

Design Document

Overview: This project implements a distributed order management system for a restaurant using CORBA. The system allows customers to view a menu, place orders, and check order status, while a manager can view all current orders. The goal is to demonstrate remote method invocation and client-server communication in a distributed environment.

System Architecture: The system follows a client-server architecture using CORBA middleware.

- Server: Hosts the order management service and processes all requests.
- Client: Provides a user interface for customers and the manager.
- ORB: Handles communication between client and server.
- Naming Service (orbd): Allows clients to locate the server object.
- Multiple clients can connect to the server simultaneously. The server maintains all order data centrally.

IDL Interface Design: A single IDL interface is used to define all operations:

- viewMenu(): Returns available items and prices
- placeOrder(username, chickenQty, colaQty): Places an order
- checkOrderStatus(username): Retrieves a specific user's order
- viewAllOrders(): Returns all orders (manager view)

Custom exceptions are defined:

- InvalidOrder: Handles invalid input
- OrderNotFound: Handles missing orders

Data Design: The server stores all orders using a HashMap:

- Key: Username
- Value: Order object

Each Order contains:

- Username
- Quantity of fried chicken
- Quantity of cola
- Total price
- This allows efficient lookup and storage of orders.

Client Design: The client provides a menu-driven interface that supports two roles:

- Customer:
 - View menu
 - Place order
 - Check order status
- Manager:
 - View all orders
- The client communicates with the server using CORBA remote method calls.

Server Design: The server implements the IDL interface and handles all business logic:

- Validates input data

- Calculates total price
- Stores and retrieves orders
- Returns results or exceptions
- The server runs continuously and listens for client requests through the ORB.

Error Handling: The system includes error handling for:

- Invalid input (negative quantities)
- Non-existent users
- Communication errors
- Exceptions are defined in IDL and handled on both client and server sides.

Performance Measurements: To evaluate system efficiency, response time for each operation was measured using `System.nanoTime()`.

The following operations were tested:

- `viewMenu`
- `placeOrder`
- `checkOrderStatus`
- `viewAllOrders`

Each operation was executed multiple times, and the average response time was calculated. Results show that simple operations have lower latency, while operations involving more data processing take slightly longer.

Conclusion: This project demonstrates how CORBA can be used to build a distributed application with multiple clients and a centralized server. The system successfully supports remote communication, data management, and role-based functionality. The design is scalable and can be extended with additional features such as persistent storage or a graphical user interface.